

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3 – Vulnerability Detection

Course Code:	CSA5802	Course Title:	DEVSECOPS ENGINEERING AND APPLICATION SECURITY
CO Assessed:	CO3 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 3
Weightage:	40% of CIA	Total Marks:	100
CO 3 Statement:	Vulnerability Detection, Severity Analysis, Security Verification Workflow Simulation and Performance Evaluation		
Student Name:	Krithika M	Reg. No.:	192521385
Section / Batch:	B-tech - IT	Date:	

Instructions to Students

- Perform vulnerability detection and classify the identified vulnerabilities based on their severity levels.
- Conduct severity analysis using appropriate metrics such as CVSS and document the impact of each vulnerability.
- Design and simulate a Security Verification Workflow covering detection, remediation, re-testing, and verification.
- Evaluate security and system performance before and after mitigation, and include relevant results, screenshots, and observations.

Question Type	Focus Area	Questions	Marks
Vulnerability Detection	Conceptual Understanding	Q1	Q1 –100
GRAND TOTAL:		100 Marks	

Code of Conduct

I Krithika M 192521385(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Krithika M

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

AI-POWERED VULNERABILITY DETECTION AND SECURE CODE REVIEW PLATFORM

1. Real-World Problem

Problem:

An online shopping company develops a web-based **E-Commerce Application** where customers can create accounts, log in, search for products, add products to a cart, and place orders.

The application is developed using Python and contains several modules such as:

- User Registration
- User Login
- Product Search
- Shopping Cart
- Order Management
- Payment Processing

Before deploying a new version of the application, developers need to ensure that the source code does not contain security vulnerabilities.

Manual code review can be time-consuming and may fail to identify hidden security issues. Therefore, an **AI-Powered Vulnerability Detection and Secure Code Review Platform** is proposed.

The platform accepts source code from developers and automatically analyzes it using security rules and AI-based code analysis techniques.

The platform can identify vulnerabilities such as:

- SQL Injection
- Cross-Site Scripting (XSS)
- Hardcoded Passwords
- Weak Password Hashing

- Command Injection
- Insecure Input Validation
- Use of insecure functions

The system then classifies each vulnerability according to its severity and provides recommendations for fixing the problem.

2. Aim

To develop and simulate an **AI-Powered Vulnerability Detection and Secure Code Review Platform** that automatically analyzes source code, identifies security vulnerabilities, classifies their severity, provides secure coding recommendations, and verifies whether the vulnerabilities have been successfully removed after remediation.

3. Objectives

The main objectives of the proposed system are:

1. To automatically analyze source code for security vulnerabilities.
 2. To identify insecure coding practices.
 3. To classify detected vulnerabilities based on severity.
 4. To explain the impact of each vulnerability.
 5. To provide secure coding recommendations.
 6. To assist developers during the code review process.
 7. To perform security verification after fixes are applied.
 8. To re-test the modified source code.
 9. To compare the security status before and after remediation.
 10. To reduce the time required for manual security code review.
-

4. Real-World Application Scenario

Consider an online shopping application that contains a **user login module**.

The developer writes the following code:

```
import sqlite3
```

```
def login(username, password):
```

```
    conn = sqlite3.connect("shop.db")
```

```
    query = "SELECT * FROM users WHERE username='" + username + \"  
            \" AND password='" + password + \"\""
```

```
    result = conn.execute(query)
```

```
    return result.fetchone()
```

The code appears to perform a normal login operation. However, the application directly combines user input with an SQL query.

This creates a potential **SQL Injection vulnerability**.

An attacker could manipulate the input supplied to the application and potentially alter the intended database query.

Therefore, the source code must be analyzed before the application is deployed.

5. Synthetic Data

The following data is completely fictional and is created only for security testing.

User ID	Username	Role	Status
U1001	arun123	Customer	Active
U1002	meena456	Customer	Active
U1003	admin01	Administrator	Active
U1004	ravi789	Customer	Active
U1005	priya321	Customer	Active

The data does not represent real users or real customer information.

Synthetic Source Code

The following source code is intentionally created with security weaknesses for demonstrating the vulnerability detection process.

```
import sqlite3
import hashlib
import os

API_KEY = "TEST_API_KEY_12345"
ADMIN_PASSWORD = "Admin@123"

def login(username, password):

    conn = sqlite3.connect("shop.db")

    query = "SELECT * FROM users WHERE username='" + username + \
        "' AND password='" + password + "'"

    result = conn.execute(query)

    return result.fetchone()

def search_product(product):

    command = "grep " + product + " products.txt"

    os.system(command)

def create_user(password):

    hashed = hashlib.md5(password.encode()).hexdigest()

    return hashed
```

The code contains multiple intentionally insecure practices.

6. Vulnerability Detection

The AI-powered platform scans the submitted source code and identifies suspicious coding patterns.

The AI code analyzer examines:

- User input handling
- Database queries
- Password storage
- Cryptographic functions
- Operating system commands
- Hardcoded credentials
- API keys
- Authentication logic

After analyzing the source code, the platform generates a vulnerability report.

7. Detected Vulnerabilities

ID	Vulnerability	Severity	Location
V01	SQL Injection	HIGH	login()
V02	Hardcoded Password	HIGH	ADMIN_PASSWORD
V03	Hardcoded API Key	HIGH	API_KEY
V04	Weak MD5 Hashing	MEDIUM	create_user()

ID	Vulnerability	Severity	Location
V05	Command Injection	Risk HIGH	search_product()

8. Vulnerability 1 – SQL Injection

Vulnerable Code:

```
query = "SELECT * FROM users WHERE username='" + username + \"  
      \" AND password='" + password + \""
```

```
result = conn.execute(query)
```

Detection:

The AI analyzer detects that user-controlled values are directly concatenated into an SQL query.

Vulnerability:

SQL Injection

Severity:

HIGH

Impact:

An attacker may manipulate database queries and potentially access or modify unauthorized information.

Reason:

The application does not safely separate user input from the SQL command.

AI Recommendation:

Use parameterized queries or prepared statements.

Secure Code:

```
query = \"  
SELECT * FROM users  
WHERE username=? AND password=?  
\"
```

```
result = conn.execute(query, (username, password))
```

This prevents user input from being directly interpreted as part of the SQL command.

9. Vulnerability 2 – Hardcoded Password

Vulnerable Code:

```
ADMIN_PASSWORD = "Admin@123"
```

Detection:

The AI analyzer identifies a password stored directly inside the source code.

Vulnerability:

Hardcoded Password

Severity:

HIGH

Impact:

If the source code is exposed through a repository or unauthorized access, the password may also be exposed.

AI Recommendation:

Do not store passwords directly in source code. Use secure authentication mechanisms and protected configuration or environment-based secrets.

Secure Approach:

```
ADMIN_PASSWORD = os.getenv("ADMIN_PASSWORD")
```

The password can then be supplied through a protected environment configuration rather than being embedded in the source code.

10. Vulnerability 3 – Hardcoded API Key

Vulnerable Code:

```
API_KEY = "TEST_API_KEY_12345"
```

Detection:

The AI analyzer detects a string that follows the pattern of an API credential.

Vulnerability:

Hardcoded API Key

Severity:**HIGH****Impact:**

If the source code is uploaded to a public repository, the API key could be exposed.

AI Recommendation:

Store sensitive credentials outside the source code.

Secure Code:

```
API_KEY = os.getenv("API_KEY")
```

This reduces the possibility of accidentally exposing credentials through the source repository.

11. Vulnerability 4 – Weak Password Hashing**Vulnerable Code:**

```
hashed = hashlib.md5(password.encode()).hexdigest()
```

Detection:

The AI analyzer identifies the use of the MD5 hashing algorithm for password protection.

Vulnerability:**Weak Password Hashing****Severity:****MEDIUM****Impact:**

MD5 is not suitable for securely storing passwords because it is a fast hashing algorithm and is considered unsuitable for modern password storage.

AI Recommendation:

Use a password-specific hashing mechanism with an appropriate salt and work factor.

Secure Approach:

```
from werkzeug.security import generate_password_hash
```

```
hashed = generate_password_hash(password)
```

12. Vulnerability 5 – Command Injection Risk**Vulnerable Code:**

```
command = "grep " + product + " products.txt"
```

```
os.system(command)
```

Detection:

The AI analyzer identifies that user-controlled input is directly added to an operating-system command.

Vulnerability:**Command Injection Risk****Severity:****HIGH****Impact:**

Improperly controlled input could cause unintended operating-system commands to be executed.

AI Recommendation:

Avoid constructing shell commands from raw user input. Use safe library functions and validate input where necessary.

Secure Approach:

Use Python file-processing functions instead of constructing shell commands from user input.

13. AI-Based Security Report

After scanning the application, the platform generates an automated report.

Sample Report:

```
=====
AI-POWERED SECURITY CODE REVIEW
=====
```

Project: E-Commerce Application

Scan Status: Completed

V01 - SQL Injection
Severity: HIGH
Location: login()
Status: OPEN

V02 - Hardcoded Password
Severity: HIGH
Location: ADMIN_PASSWORD
Status: OPEN

V03 - Hardcoded API Key
Severity: HIGH
Location: API_KEY
Status: OPEN

V04 - Weak Password Hashing
Severity: MEDIUM
Location: create_user()
Status: OPEN

V05 - Command Injection Risk
Severity: HIGH
Location: search_product()
Status: OPEN

Total Vulnerabilities: 5
High: 4
Medium: 1
Low: 0
=====

14. Severity Analysis

The platform assigns severity levels based on the potential impact and exploitability of the detected vulnerability.

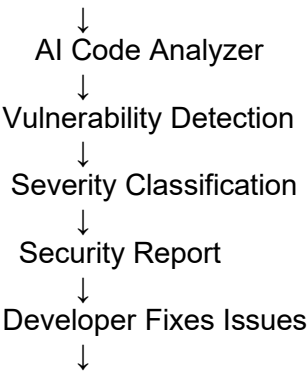
Severity	Meaning	Example
Critical	Extremely serious security risk	Remote code execution
High	Serious security risk	SQL Injection
Medium	Moderate security risk	Weak hashing
Low	Minor security weakness	Missing non-critical validation

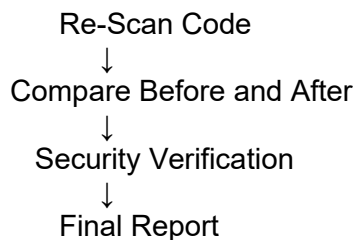
For this project, the detected vulnerabilities are mainly categorized as **High** and **Medium** severity. The severity classification helps developers decide which vulnerabilities should be fixed first.

15. Security Verification Workflow

The proposed platform follows a complete security verification workflow:

Developer Submits Source Code





This workflow ensures that vulnerability detection is not limited to identifying problems. The system also verifies whether the implemented security fixes are effective.

16. Remediation Process

The developer applies the recommendations generated by the AI platform.

Before Remediation:

- SQL queries use direct string concatenation.
- Password is hardcoded.
- API key is hardcoded.
- MD5 is used for password hashing.
- User input is used to construct an operating-system command.

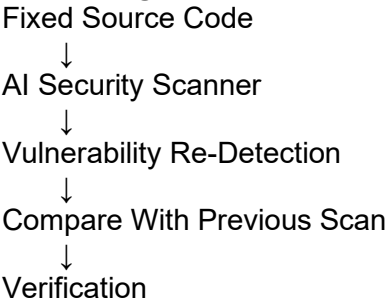
After Remediation:

- Parameterized SQL queries are used.
- Sensitive passwords are removed from source code.
- API keys are stored securely outside the source code.
- Secure password hashing is used.
- Unsafe shell command construction is removed.

17. Re-Testing

After the developer fixes the identified vulnerabilities, the source code is submitted to the platform again. The AI analyzer performs another security scan.

Re-Testing Process:



If the previously identified vulnerabilities are no longer detected, their status is changed from **OPEN** to **RESOLVED**.

18. Before and After Comparison

Vulnerability	Before Fix	After Fix
SQL Injection	Detected	Resolved
Hardcoded Password	Detected	Resolved
Hardcoded API Key	Detected	Resolved
Weak MD5 Hashing	Detected	Resolved
Command Injection Risk	Detected	Resolved

Result:

BEFORE MITIGATION

High Vulnerabilities : 4
Medium Vulnerabilities : 1
Total : 5

↓
SECURITY FIXES
↓

AFTER MITIGATION

High Vulnerabilities : 0
Medium Vulnerabilities : 0
Total : 0

19. Performance Evaluation

The security platform can also evaluate the effectiveness of automated code review.

Parameter	Before AI Review	After AI Review
Vulnerabilities Detected	0 manually identified	5 automatically detected
Security Analysis	Manual	Automated
Severity Classification	Manual	Automated
Security Recommendations	Manual	AI-assisted
Re-testing	Manual	Automated
Final Verification	Manual	Automated

The results show that the proposed platform can reduce the effort required for repetitive security code review and provide developers with faster feedback.

20. Advantages of the Proposed Platform

The proposed AI-powered platform provides the following advantages:

- 1. Automated Vulnerability Detection**
The platform automatically analyzes source code and identifies security weaknesses.
- 2. Faster Code Review**
Developers receive security feedback without waiting for a complete manual review.
- 3. Severity Classification**
Vulnerabilities are categorized according to their security impact.
- 4. AI-Based Recommendations**
The platform provides suggestions for secure coding practices.
- 5. Early Detection**
Security issues can be identified before the application reaches production.
- 6. Re-Testing**
Modified code can be scanned again to verify whether vulnerabilities have been fixed.
- 7. Security Documentation**
The platform generates a report containing detected vulnerabilities, severity, location, and recommendations.

21. Expected Output

The final platform produces a security dashboard or report containing:

=====

```
AI SECURITY CODE REVIEW PLATFORM
```

=====

Project: E-Commerce Application

SCAN COMPLETED

Total Vulnerabilities : 5
High Severity : 4
Medium Severity : 1

Low Severity : 0

Security Status : NOT SECURE

Recommendations:

- ✓ Fix SQL Injection
- ✓ Remove hardcoded credentials
- ✓ Protect API keys
- ✓ Replace weak password hashing
- ✓ Remove unsafe command execution

=====

After remediation and re-testing:

=====

FINAL SECURITY VERIFICATION

=====

Previous Vulnerabilities : 5

Resolved Vulnerabilities : 5

Remaining Vulnerabilities: 0

Security Status: SECURE

Verification: PASSED

=====

22. Result

Thus, the **AI-Powered Vulnerability Detection and Secure Code Review Platform** was successfully designed and simulated using synthetic source code. The platform identified important security vulnerabilities including **SQL Injection, hardcoded passwords, hardcoded API keys, weak password hashing, and command injection risk**.

The detected vulnerabilities were classified according to their severity, and appropriate secure coding recommendations were provided. After applying the recommended security fixes, the source code was re-tested and the vulnerabilities were verified as resolved.

The proposed system demonstrates how AI-assisted security analysis can support developers by automating vulnerability detection, severity analysis, secure code recommendations, remediation tracking, re-testing, and final security verification.

Final Workflow

Synthetic Source Code



Code Submission



AI-Powered Code Analyzer



Vulnerability Detection



Severity Analysis



Security Report



Secure Code Recommendations



Developer Remediation



AI Re-Scan



Before/After Comparison



Security Verification



Final Secure Code

This completes the simulation of the **AI-Powered Vulnerability Detection and Secure Code Review Platform** for a real-world e-commerce application scenario.